



دانشگاه تهران

دانشکده مهندسی برق و کامپیوتر

درس مبانی رایانش توزیع شده
گزارش تمرین دوم

دانشجویان:

سارا گیتی ۸۱۰۱۰۱۵۰۰

پریا پاسه‌ورز ۸۱۰۱۰۱۳۹۳

فهرست مطالب

۵	فهرست تصاویر
۶	فهرست جداول
۷	۱ بخش اول: آشنایی با RPC و فناوری‌های مختلف آن
۷	۱-۱ شرح بخش
۷	۲-۱ تعریف RPC
۷	۳-۱ ایده اصلی RPC
۹	۴-۱ تفاوت فراخوانی محلی و فراخوانی راه دور
۹	۱-۴-۱ فراخوانی محلی (Local Procedure Call)
۹	۲-۴-۱ فراخوانی راه دور (Remote Procedure Call)
۱۰	۵-۱ نقش Client Stub و Server Stub
۱۰	۱-۵-۱ Client Stub
۱۰	۲-۵-۱ Server Stub
۱۱	۶-۱ معماری RPC
۱۲	۷-۱ مفهوم Serialization و Deserialization
۱۳	۱-۷-۱ مثال با Pickle Python
۱۳	۸-۱ مفهوم IDL (Interface Definition Language)
۱۴	۹-۱ خطاهای رایج در RPC
۱۵	۱-۹-۱ خطای عدم دسترسی به سرور
۱۶	۲-۹-۱ خطای از دست رفتن پیام (Message Loss)
۱۷	۳-۹-۱ ناسازگاری نوع داده‌ها
۱۷	۴-۹-۱ مشکل Pointerها و Referenceها
۱۸	۱۰-۱ انواع RPC
۱۸	۱-۱۰-۱ RPC همگام (Synchronous RPC)

۱۹	RPC ناهمگام (Asynchronous RPC)
۲۰	Deferred Synchronous RPC
۲۰	One-way RPC
۲۱	Multicast RPC
۲۱	مقایسه RPC با REST
۲۲	REST چیست؟
۲۲	تفاوت REST و RPC
۲۲	مقایسه چند فناوری رایج RPC
۲۲	gRPC
۲۴	JSON-RPC
۲۵	XML-RPC
۲۶	Apache Thrift
۲۶	جدول مقایسه فناوری‌های RPC
۲۷	پاسخ به پرسش‌های تحلیلی
	۱-۱۴ پرسش ۱: چرا RPC باعث می‌شود فراخوانی یک سرویس راه دور شبیه
۲۷	فراخوانی یک تابع محلی به نظر برسد؟
۲۷	۲-۱۴ پرسش ۲: چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟
	۳-۱۴ پرسش ۳: چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که
۲۸	در فراخوانی محلی وجود ندارد؟
۲۹	۴-۱۴ پرسش ۴: در چه شرایطی gRPC انتخاب بهتری از REST است؟
۲۹	۵-۱۴ پرسش ۵: در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟
۳۰	۱۵-۱ مثال واقعی RPC با gRPC در Go
۳۰	۱-۱۵ پروتو فایل
۳۱	۲-۱۵ سمت Server
۳۱	۳-۱۵ سمت Client
۳۲	۱۶-۱ نتیجه‌گیری نهایی

٣٢ ١٧-١ منابع

فهرست تصاویر

فهرست جداول

۲۳	مقایسه فراخوانی محلی و راه دور	۱
۲۳	مقایسه REST و RPC	۲
۲۶	جدول مقایسه فناوری‌های RPC	۳

۱ بخش اول: آشنایی با RPC و فناوری‌های مختلف آن

۱-۱ شرح بخش

در این بخش، یک گزارش کامل درباره RPC تهیه شده است. این گزارش دقیق، فنی و همراه با مقایسه فناوری‌های مختلف RPC است.

۲-۱ تعریف RPC

RPC مخفف Remote Procedure Call و به معنی فراخوانی رویه یا تابع از راه دور است. RPC یک مدل ارتباطی در سیستم‌های توزیع‌شده است که به یک برنامه اجازه می‌دهد تابعی را که روی یک کامپیوتر دیگر قرار دارد، همانند یک تابع محلی فراخوانی کند. به‌صورت ساده، در RPC برنامه کلاینت (Client) تابعی را فراخوانی می‌کند، اما اجرای واقعی تابع در یک ماشین دیگر (Server) صورت می‌گیرد. با این حال، این تفاوت برای برنامه‌نویس تا حد زیادی پنهان می‌شود.

در کل RPC را به‌عنوان مکانیزمی معرفی می‌شود که هدف آن پنهان‌سازی جزئیات ارسال و دریافت پیام‌های شبکه‌ای است. به جای آنکه برنامه‌نویس مستقیماً از توابع `send()` و `receive()` استفاده کند، تنها یک تابع را فراخوانی می‌کند؛ در حالی که سیستم RPC تمام جزئیات ارتباط را مدیریت می‌کند.

تعریف ۱.۱. RPC روشی است که در آن برنامه‌نویس می‌تواند تابعی را که روی سیستم دیگری اجرا می‌شود، همانند یک تابع محلی فراخوانی کند، بدون آنکه درگیر جزئیات پیچیده ارتباط شبکه‌ای شود.

۳-۱ ایده اصلی RPC

ایده اصلی RPC بر پایه ایجاد شفافیت در فراخوانی توابع از راه دور شکل گرفته است.

در حالت عادی، اگر بخواهیم دو برنامه روی دو ماشین مختلف با یکدیگر ارتباط برقرار کنند، باید:

۱. پیام مناسب ساخته شود.
۲. داده‌ها قالب‌بندی شوند.
۳. پیام از طریق شبکه ارسال شود.
۴. سمت مقابل پیام را دریافت کند.
۵. داده‌ها استخراج شوند.
۶. عملیات موردنظر اجرا شود.
۷. پاسخ مجدداً ارسال گردد.

این فرایند پیچیده و مستعد خطاست. RPC این پیچیدگی را مخفی می‌کند. در RPC، یک فراخوانی محلی تابع در سمت کلاینت به مجموعه‌ای از عملیات شبکه‌ای تبدیل می‌شود که در نهایت باعث اجرای همان تابع در سمت سرور می‌گردد، بدون آنکه کلاینت از این جزئیات آگاه باشد.

مثال ۲.۱. به جای نوشتن کد پیچیده شبکه:

```
1 send("GET_BALANCE", user_id)
2 response = receive()
```

برنامه‌نویس تنها می‌نویسد:

```
1 balance = getBalance(user_id)
```

در ظاهر این یک فراخوانی تابع عادی است؛ اما در پشت صحنه، سیستم RPC درخواست را به سرور ارسال کرده، تابع را اجرا می‌کند و نتیجه را بازمی‌گرداند.

۴-۱ تفاوت فراخوانی محلی و فراخوانی راه دور

یکی از موضوعات مهم در درک RPC، فهم تفاوت میان فراخوانی محلی و فراخوانی راه دور است.

۱-۴-۱ فراخوانی محلی (Local Procedure Call)

در فراخوانی محلی، تابع موردنظر در همان فضای حافظه و روی همان سیستم اجرا می‌شود.

```
1 sum = add(5, 7)
```

در اینجا:

- تابع روی همان سیستم اجرا می‌شود.
- دسترسی مستقیم به حافظه وجود دارد.
- هزینه زمانی کم است.
- نیازی به شبکه نیست.

۲-۴-۱ فراخوانی راه دور (Remote Procedure Call)

در RPC، تابع روی سیستم دیگری اجرا می‌شود.

```
1 user = getUserInfo(15)
```

ممکن است این تابع در واقع روی یک سرور در کشور دیگری اجرا شود.
در پشت صحنه:

۱. پارامترها بسته‌بندی می‌شوند.
۲. پیام از طریق شبکه ارسال می‌شود.
۳. سرور تابع را اجرا می‌کند.

۴. نتیجه بازگردانده می‌شود.

اما برای برنامه‌نویس همه چیز شبیه یک فراخوانی ساده به نظر می‌رسد. یکی از نکات مهم این است که RPC تلاش می‌کند تجربه فراخوانی محلی را شبیه‌سازی کند، اما هرگز دقیقاً همان نیست؛ زیرا محدودیت‌های شبکه، تأخیر، خرابی سیستم و ناسازگاری ماشین‌ها همواره وجود دارند.

۵-۱ نقش Server Stub و Client Stub

یکی از مهم‌ترین مفاهیم RPC، Stub است.

۱-۵-۱ Client Stub

Client Stub واسطه‌ای میان برنامه کلاینت و شبکه است. وظایف آن:

- دریافت فراخوانی تابع
 - بسته‌بندی پارامترها (Marshaling)
 - ارسال درخواست
 - انتظار برای پاسخ
 - بازگرداندن نتیجه
- Client Stub وظیفه دارد فراخوانی عادی تابع را به یک پیام شبکه‌ای تبدیل کند.

۲-۵-۱ Server Stub

Server Stub نقش معکوس دارد. وظایف:

- دریافت پیام
 - استخراج پارامترها (Unmarshaling)
 - فراخوانی تابع واقعی
 - آماده‌سازی پاسخ
- Server Stub درخواست شبکه‌ای را به یک فراخوانی محلی تابع تبدیل می‌کند؛ به‌طوری‌که از دید تابع سمت سرور، انگار یک فراخوانی عادی انجام شده است.

۶-۱ معماری RPC

معماری RPC معمولاً شامل اجزای زیر است:

Client •

Client Stub •

Network •

Server Stub •

Server •

روند اجرای RPC به‌صورت زیر است:

۱. کلاینت تابع را فراخوانی می‌کند.
۲. Client Stub پیام را می‌سازد.
۳. سیستم‌عامل پیام را از طریق شبکه ارسال می‌کند.
۴. پیام به Server Stub تحویل داده می‌شود.
۵. پارامترها استخراج می‌شوند.

۶. تابع واقعی اجرا می‌شود.

۷. نتیجه بسته‌بندی می‌شود.

۸. پاسخ به کلاینت بازگردانده می‌شود.

۹. Client Stub پاسخ را دریافت می‌کند.

۱۰. نتیجه به برنامه تحویل داده می‌شود.

۷-۱ مفهوم Serialization و Deserialization

یکی از مهم‌ترین مشکلات RPC انتقال داده بین ماشین‌های مختلف است. ماشین‌ها ممکن است:

- ساختار حافظه متفاوت داشته باشند

- ترتیب بایت متفاوت (Endianness) داشته باشند

- قالب ذخیره‌سازی متفاوتی داشته باشند

برای حل این مشکل از **Marshaling** و **Unmarshaling** استفاده می‌شود.

تعریف ۳.۱. Serialization (یا Marshaling): تبدیل داده به فرمت قابل انتقال.

مثال ۴.۱. تبدیل داده به JSON:

```
1 {  
2   "id": 10,  
3   "name": "Ali"  
4 }
```

تعریف ۵.۱. Deserialization (یا Unmarshaling): تبدیل داده دریافتی به شیء قابل استفاده.

۱-۷-۱ مثال با Pickle Python

نمونه‌ای از RPC در Python که از کتابخانه pickle برای Serialization استفاده می‌کند.

نمونه کد:

```
1 msgsnd = pickle.dumps(msglst)
2 retval = pickle.loads(msgrcv[1])
```

در اینجا:

- dumps() داده را serialize می‌کند.

- loads() داده را deserialize می‌کند.

۸-۱ مفهوم IDL (Interface Definition Language)

یکی از مشکلات RPC آن است که Client و Server باید دقیقاً بدانند:

- چه تابعی وجود دارد

- پارامترها چیست

- نوع داده‌ها چیست

برای این کار از IDL استفاده می‌شود.

IDL مخفف **Language Definition Interface** است.

IDL به توسعه‌دهنده اجازه می‌دهد رابط‌های RPC را تعریف کند و سپس سیستم به صورت

خودکار Client Stub و Server Stub را تولید نماید.

مثال ۶.۱. نمونه در gRPC:

```
1 syntax = "proto3";
2
3 service UserService {
```

```
4   rpc GetUser(UserRequest) returns (UserResponse);
5 }
6
7 message UserRequest {
8     int32 id = 1;
9 }
10
11 message UserResponse {
12     int32 id = 1;
13     string name = 2;
14 }
```

در اینجا:

• GetUser تابع راه دور است.

• UserRequest ورودی است.

• UserResponse خروجی است.

سپس gRPC به صورت خودکار کد Client و Server را تولید می‌کند.

۹-۱ خطاهای رایج در RPC

اگرچه RPC تلاش می‌کند فراخوانی تابع از راه دور را مشابه فراخوانی محلی جلوه دهد، اما در عمل تفاوت‌های مهمی میان این دو وجود دارد. یکی از اساسی‌ترین محدودیت‌های RPC این است که برخلاف فراخوانی محلی، ارتباط از طریق شبکه انجام می‌شود و شبکه ذاتاً غیرقابل اعتمادتر، کندتر و مستعد خطاست. در فراخوانی محلی، معمولاً فرض می‌شود که:

- حافظه در دسترس است،

- تابع وجود دارد،

- سیستم فعال است،

- و ارتباط تقریباً آنی است.

اما در RPC این فرض‌ها همیشه برقرار نیستند.

در کل RPC هرگز کاملاً معادل Local Procedure Call نیست، زیرا در محیط توزیع‌شده عواملی مانند خرابی شبکه، قطع ارتباط، تأخیر و ناسازگاری سیستم‌ها وجود دارند.

۱-۹-۱ خطای عدم دسترسی به سرور

یکی از رایج‌ترین مشکلات RPC زمانی رخ می‌دهد که سرور در دسترس نباشد.
برای مثال:

- سرور خاموش شده باشد،

- شبکه قطع شده باشد،

- یا سرویس RPC کرش کرده باشد.

در چنین شرایطی، کلاینت درخواست را ارسال می‌کند اما پاسخی دریافت نمی‌شود.

مثال ۷.۱. فرض کنید برنامه‌ای داریم:

```
1 user = getUser(15)
```

اگر سرور خاموش باشد، چه اتفاقی می‌افتد؟

در Local Call چنین مسئله‌ای تقریباً وجود ندارد، اما در RPC باید:

- Timeout تعریف شود،

- Retry mechanism وجود داشته باشد،

• Exception handling مناسب انجام شود.

```
1 try:
2     user = client.get_user(15)
3 except TimeoutError:
4     print("Server unavailable")
```

۱-۹-۲ خطای از دست رفتن پیام (Message Loss)

در شبکه ممکن است پیام:

• گم شود،

• دیر برسد،

• یا چند بار ارسال شود.

این موضوع در RPC بسیار مهم است، زیرا کلاینت ممکن است نداند آیا درخواست اجرا شده یا خیر.

مثال ۸.۱. نمونه Message Loss:

```
1 transferMoney(1000)
```

اگر پاسخ دریافت نشود، آیا پول منتقل شده است؟

سه حالت وجود دارد:

۱. درخواست هرگز نرسیده است.

۲. درخواست اجرا شده اما پاسخ گم شده است.

۳. سرور در میانه پردازش از کار افتاده است.

به همین دلیل RPC Framework ها سیاست‌هایی مانند:

• At-most-once

• At-least-once

• Exactly-once (تقریباً غیرممکن در عمل)

را پیاده‌سازی می‌کنند.

۳-۹-۱ ناسازگاری نوع داده‌ها

ماشین‌های مختلف ممکن است:

• معماری متفاوت داشته باشند،

• Endianness متفاوت داشته باشند،

• قالب ذخیره‌سازی متفاوتی برای اعداد داشته باشند.

با توجه به مسئله **Endian Big** و **Endian Little**، سیستم RPC باید داده‌ها را به فرمتی مستقل از ماشین تبدیل کند. به همین دلیل Serialization ضروری است.

۴-۹-۱ مشکل Pointerها و Referenceها

یکی از دشوارترین مسائل RPC انتقال Pointer است.

در کل Pointer فقط در فضای حافظه همان فرایند معنا دارد. بنابراین نمی‌توان آدرس حافظه یک سیستم را مستقیماً به سیستم دیگر فرستاد.

مثال ۹.۱. نمونه مشکل Pointer:

```
۱ update(&user)
```

در سیستم محلی مشکلی وجود ندارد، اما در RPC آدرس حافظه `user` روی سرور بی‌معنا خواهد بود.

راه‌حل معمول:

• Copy-by-value

• تبدیل Object به داده قابل انتقال

۱-۱۰ انواع RPC

چند مدل مختلف از RPC را معرفی می‌شود که بسته به نیاز سیستم استفاده می‌شوند. مهم‌ترین آن‌ها عبارت‌اند از:

۱. Synchronous RPC

۲. Asynchronous RPC

۳. Deferred Synchronous RPC

۴. One-way RPC

۵. Multicast RPC

۱-۱۰-۱ RPC همگام (Synchronous RPC)

این مدل، رایج‌ترین شکل RPC است.
در این حالت:

• کلاینت درخواست را ارسال می‌کند.

• منتظر پاسخ می‌ماند.

• تا زمان دریافت پاسخ، اجرای برنامه متوقف می‌شود.

شکل کلی:

Client ---- Request ----> Server

Client <--- Response ---- Server

در این مدل رفتار RPC بسیار شبیه فراخوانی عادی تابع است.

مثال ۱۰.۱. نمونه RPC:

```
1 result = getUser(10)
2 print(result)
```

برنامه تا بازگشت نتیجه متوقف می‌شود.

مزایا:

- ساده
- قابل فهم
- مشابه Local Call

معایب:

- Blocking
 - کاهش کارایی در سیستم‌های بزرگ
- این مدل را رفتار سنتی Request-Reply است.

۱-۱۰-۲ RPC ناهمگام (Asynchronous RPC)

در این مدل کلاینت پس از ارسال درخواست متوقف نمی‌شود.
سرور تنها تأیید دریافت را ارسال می‌کند و پردازش ادامه می‌یابد.
ساختار:

```
Client ---- Request ----> Server
Client <---- ACK ----- Server
Client continues...
```

مزیت:

- افزایش کارایی

- جلوگیری از Blocking

این روش زمانی مفید است که پاسخ فوری لازم نباشد یا چندین RPC به صورت موازی اجرا شوند.

۳-۱۰-۱ Deferred Synchronous RPC

در این مدل:

۱. کلاینت درخواست را ارسال می‌کند.

۲. تأیید دریافت می‌شود.

۳. کلاینت به کار خود ادامه می‌دهد.

۴. نتیجه بعداً از طریق Callback بازگردانده می‌شود.

این روش را برای زمانی مناسب است که کلاینت بخواهد هم‌زمان با چند سرور ارتباط برقرار کند.

۴-۱۰-۱ One-way RPC

در این مدل:

- کلاینت فقط درخواست را ارسال می‌کند.

- منتظر هیچ پاسخی نمی‌ماند.

مثال ۱۱.۱. نمونه درخواست One-way RPC:

```
logActivity(user_id)
```

مزایا:

- سریع
- مناسب Logging

معایب:

- عدم اطمینان از اجرای موفق

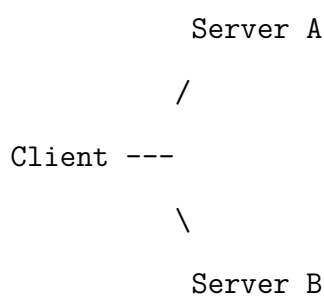
۵-۱۰-۱ Multicast RPC

مفهوم **RPC Multicast** یعنی، جایی که یک درخواست هم‌زمان به چند سرور ارسال می‌شود.

مثال کاربرد:

- Replication
- Fault tolerance
- Parallel processing

ساختار:



۱۱-۱ مقایسه REST با RPC

یکی از مهم‌ترین موضوعات در معماری سیستم‌های توزیع‌شده، مقایسه REST با RPC است.

۱-۱۱-۱ REST چیست؟

REST یک سبک معماری برای طراحی API‌های وب است که معمولاً از HTTP استفاده می‌کند.
نمونه:

GET /users/15

۲-۱۱-۱ تفاوت REST و RPC

تحلیل:

RPC معمولاً:

- سریع‌تر است،
- مناسب سرویس‌های داخلی است،
- در Microservice‌ها رایج‌تر است.

REST:

- ساده‌تر است،
- Debug آسان‌تری دارد،
- برای API عمومی بهتر است.

۱۲-۱ مقایسه چند فناوری رایج RPC

۱-۱۲-۱ gRPC

gRPC یکی از مدرن‌ترین Framework‌های RPC است که توسط Google توسعه یافته است.
ویژگی‌ها:

جدول ۱: مقایسه فراخوانی محلی و راه دور

ویژگی	فراخوانی محلی	فراخوانی راه دور
محل اجرا	همان سیستم	سیستم راه دور
نیاز به شبکه	ندارد	دارد
سرعت	بالا	کمتر
احتمال خطا	کم	بیشتر
Serialization	نیاز ندارد	نیاز دارد
پیچیدگی ارتباط	کم	زیاد

جدول ۲: مقایسه *RPC* و *REST*

ویژگی	<i>RPC</i>	<i>REST</i>
مدل	تابع محور	منبع محور
کارایی	بالا تر	کمتر
خوانایی	کمتر	بیشتر
سادگی Debug	سخت تر	آسان تر
مناسب Microservice	بسیار مناسب	مناسب
پروتکل	TCP, gRPC	HTTP

- استفاده از HTTP/2
- سرعت بالا
- پشتیبانی از Streaming
- استفاده از Protocol Buffers

مثال ۱۲.۱. نمونه IDL:

```
1 service UserService {  
2     rpc GetUser(UserRequest) returns (UserResponse);  
3 }
```

مزایا:

- بسیار سریع
- مناسب Microservice
- Typed interface
- معایب:**
- Debug سخت‌تر
- خوانایی کمتر نسبت به REST

۲-۱۲-۱ JSON-RPC

RPC مبتنی بر JSON است.

مثال ۱۳.۱. نمونه JSON:

```
1 {  
2   "jsonrpc": "2.0",  
3   "method": "subtract",  
4   "params": [42,23],  
5   "id": 1  
6 }
```

مزایا:

- ساده
- خوانا

معایب:

- کندتر از gRPC

۱-۱۲-۳ XML-RPC

یکی از قدیمی‌ترین فناوری‌های RPC است.

مثال ۱۴.۱. نمونه XML-RPC:

```
1 <methodCall>  
2   <methodName>getUser</methodName>  
3 </methodCall>
```

مزایا:

- استاندارد قدیمی و پایدار

معایب:

- حجیم و کند

۴-۱۲-۱ Apache Thrift

فریم‌ورک RPC توسعه داده شده توسط Facebook.

ویژگی‌ها:

• Multi-language

• High performance

• Code generation

۱۳-۱ جدول مقایسه فناوری‌های RPC

جدول ۳: جدول مقایسه فناوری‌های RPC

فناوری	فرمت داده	IDL	موارد استفاده رایج	Streaming
gRPC	Protocol Buffers (باینری)	بله	Microservices، سیستم‌های پرتراфик	بله (دو طرفه)
JSON-RPC	JSON (متنی)	خیر/اختیاری	API‌های ساده، پروژه‌های کوچک	خیر
XML-RPC	XML (متنی)	خیر	سیستم‌های قدیمی (Legacy)	خیر
Apache Thrift	باینری (قابل تنظیم)	بله	سیستم‌های ناهمگون، نیاز به انعطاف بالا	محدود

۱۴-۱ پاسخ به پرسش‌های تحلیلی

در این بخش، به پرسش‌های تحلیلی مطرح‌شده در تمرین پاسخ داده می‌شود. هدف این قسمت صرفاً ارائهٔ پاسخ کوتاه نیست، بلکه تحلیل فنی و استدلال مهندسی در مورد انتخاب فناوری RPC مناسب در سناریوهای مختلف است.

۱-۱۴-۱ پرسش ۱: چرا RPC باعث می‌شود فراخوانی یک سرویس راه دور شبیه فراخوانی یک تابع محلی به نظر برسد؟

RPC با استفاده از مکانیزم **Stub** این شبیه‌سازی را انجام می‌دهد. **Client Stub** همان توابع محلی را در اختیار برنامه‌نویس قرار می‌دهد. وقتی برنامه‌نویس تابعی مانند `GetUser(15)` را فراخوانی می‌کند، در پشت صحنه **Stub** کارهای زیر را انجام می‌دهد:

۱. پارامترها را **Serialize** می‌کند.

۲. درخواست را از طریق شبکه ارسال می‌کند.

۳. منتظر پاسخ می‌ماند.

۴. پاسخ را **Deserialize** کرده و نتیجه را برمی‌گرداند.

از دید برنامه‌نویس، فراخوانی `GetUser(15)` کاملاً شبیه یک تابع محلی است. به این ویژگی **Location Transparency** یا شفافیت مکان می‌گویند.

۱-۱۴-۲ پرسش ۲: چرا این شباهت می‌تواند خطرناک یا گمراه‌کننده باشد؟

این شباهت خطرناک است زیرا فراخوانی محلی و راه دور ذاتاً تفاوت‌های بنیادی دارند:

۱. **زمان اجرا:** فراخوانی محلی در نانو ثانیه، اما فراخوانی راه دور در میلی‌ثانیه تا ثانیه (به خاطر تأخیر شبکه).

۲. **خطاها:** در فراخوانی محلی خطاهایی مانند Timeout، قطع شدن شبکه یا خرابی سرور معنا ندارند.

۳. **اشتراک حافظه:** فراخوانی محلی به حافظه مشترک دسترسی دارد (اشاره‌گرها کار می‌کنند)، اما در RPC نمی‌توان اشاره‌گر ارسال کرد.

خطر اصلی: برنامه‌نویس ممکن است فکر کند کار ساده است و برای خطاهای شبکه، Timeout و Partial Failure برنامه‌ای ننویسد. در نتیجه سیستم در شرایط واقعی توزیع‌شده به درستی کار نمی‌کند.

در کل RPC هرگز کاملاً معادل Local Procedure Call نیست. همچنان با توجه به جمله معروف “Network is not transparent”.

۱-۱۴-۳ پرسش ۳: چه خطاهایی در فراخوانی راه دور ممکن است رخ دهد که در فراخوانی محلی وجود ندارد؟

۱. **خطای عدم دسترسی به سرور:** سرور خاموش باشد، شبکه قطع باشد یا سرویس کرش کرده باشد.

۲. **Timeout:** سرور پاسخ ندهد (به دلیل شلوغی یا قفل شدن).

۳. **از دست رفتن پیام (Message Loss):** درخواست یا پاسخ در شبکه گم شود.

۴. **Partial Failure:** درخواست به سرور برسد و پردازش شروع شود، اما وسط کار سرور کرش کند. کلاینت نمی‌داند عملیات انجام شده یا نه.

۵. **ناسازگاری داده‌ها (Serialization Mismatch):** مثلاً سمت کلاینت یک فیلد جدید داشته باشد که سرور نمی‌شناسد.

۶. **مشکل Idempotency:** اگر درخواست دوباره فرستاده شود (به خاطر Timeout)، ممکن است عملیات دو بار انجام شود (مثل انتقال پول).

۷. **مشکل Pointerها:** اشاره‌گرها فقط در فضای حافظه همان فرایند معنا دارند.

۱-۱۴-۴ پرسش ۴: در چه شرایطی gRPC انتخاب بهتری از REST است؟

gRPC در شرایط زیر بهتر است:

۱. ارتباط داخلی Microservice ها: سرعت بالا و سربار کم برای ارتباط بین سرویس‌ها حیاتی است.
۲. نیاز به Streaming: gRPC از Streaming دو طرفه پشتیبانی می‌کند (مثل سیستم‌های چت یا مانیتورینگ لحظه‌ای).
۳. سیستم‌های چندزبانه (Polyglot): gRPC با تولید خودکار کد برای زبان‌های مختلف، هماهنگی را تضمین می‌کند.
۴. نیاز به کارایی بالا: فرمت باینری Protobuf نسبت به JSON حجم کمتری دارد و سریع‌تر است. در سیستم‌هایی که هزاران درخواست در ثانیه دارند، این تفاوت بسیار مهم است.
۵. معماری Contract-First: وقتی بخواهیم قرارداد سرویس را دقیقاً مشخص کنیم.
۶. استفاده از HTTP/2: قابلیت‌هایی مانند Multiplexing، Header Compression و Persistent Connections که تأخیر شبکه را کاهش می‌دهند.

۱-۱۴-۵ پرسش ۵: در چه شرایطی REST انتخاب ساده‌تر یا مناسب‌تر است؟

REST در شرایط زیر مناسب‌تر است:

۱. API عمومی (Public API): مرورگرها و کلاینت‌های مختلف به راحتی با JSON کار می‌کنند. gRPC نیازمند ابزارهای خاص است.
۲. پروژه‌های کوچک و سریع: نوشتن یک API با Flask یا Express بسیار سریع‌تر از تعریف فایل Protobuf و تولید کد است.
۳. نیاز به Caching: REST در سطح HTTP کش شدن را پشتیبانی می‌کند، gRPC خیر.

۴. سیستم‌های ساده CRUD: کارهایی که فقط شامل ایجاد، خواندن، به‌روزرسانی و حذف منابع هستند. REST با متدهای GET، POST، PUT، DELETE طبیعی‌تر و خواناتر است.
۵. دیباگ کردن: می‌توان با curl یا Postman مستقیماً درخواست فرستاد و پاسخ JSON را دید. gRPC به ابزار مخصوص (grpcurl) نیاز دارد.
۶. سازگاری بیشتر: REST تقریباً روی همه پلتفرم‌ها کار می‌کند، اما gRPC گاهی با Firewall، Proxy یا مرورگر مشکل دارد.

۱۵-۱ مثال واقعی RPC با gRPC در Go

با توجه به اینکه بخش عملی پروژه با Go انجام می‌شود، استفاده از مثال gRPC در Go می‌تواند کیفیت گزارش را افزایش دهد.

۱-۱۵-۱ فایل proto

```
1 syntax = "proto3";
2
3 package user;
4
5 service UserService {
6     rpc GetUser(UserRequest) returns (UserResponse);
7 }
8
9 message UserRequest {
10     int32 id = 1;
11 }
12
13 message UserResponse {
```

```
14     int32 id = 1;
15     string name = 2;
16 }
```

۲-۱۵-۱ سمت Server

```
1 func (s *server) GetUser(
2     ctx context.Context,
3     req *pb.UserRequest,
4 ) (*pb.UserResponse, error) {
5     return &pb.UserResponse{
6         Id:    req.Id,
7         Name:  "Ali",
8     }, nil
9 }
```

۳-۱۵-۱ سمت Client

```
1 resp, err := client.GetUser(
2     context.Background(),
3     &pb.UserRequest{Id: 15},
4 )
5 fmt.Println(resp.Name)
```

این مثال نشان می‌دهد که از دید برنامه‌نویس:

```
1 client.GetUser()
```

فقط یک تابع عادی است، در حالی که در پشت صحنه RPC اتفاق می‌افتد.

۱۶-۱ نتیجه‌گیری نهایی

RPC یکی از بنیادی‌ترین مفاهیم سیستم‌های توزیع‌شده است که هدف اصلی آن پنهان‌سازی پیچیدگی‌های ارتباط شبکه‌ای و فراهم‌کردن شفافیت دسترسی است. در RPC، برنامه‌نویس می‌تواند توابع راه دور را مشابه توابع محلی فراخوانی کند، در حالی که جزئیات ارتباط توسط Stubها، Serialization و پروتکل‌های ارتباطی مدیریت می‌شوند. اگرچه RPC تلاش می‌کند تفاوت میان فراخوانی محلی و راه دور را مخفی کند، اما محدودیت‌هایی مانند تأخیر شبکه، خرابی سیستم، ناسازگاری داده‌ها و مشکلات انتقال Referenceها همواره وجود دارند. به همین دلیل RPC هرگز جایگزین کامل Local Call نیست. در میان فناوری‌های موجود، gRPC به دلیل کارایی بالا، استفاده از HTTP/2، پشتیبانی از Pro-tocol Buffers و قابلیت تولید خودکار کد، گزینه‌ای بسیار مناسب برای معماری‌های Microservice است. در مقابل، فناوری‌هایی مانند REST یا JSON-RPC به دلیل سادگی و خوانایی، همچنان در بسیاری از پروژه‌ها کاربرد گسترده دارند. در نهایت، انتخاب فناوری مناسب RPC وابسته به نیازهای پروژه، مقیاس سیستم، نوع ارتباط، شرایط شبکه و میزان اهمیت کارایی و سادگی توسعه است.

۱۷-۱ منابع

۱. Edith Systems, Distributed Tanenbaum, S. Andrew Steen, van Maarten
tion.

۲. <https://grpc.io/docs/>: gRPC Official Documentation

۳. <https://protobuf.dev/>: Protocol Buffers Documentation

۴. <https://thrift.apache.org/>: Apache Thrift Documentation

۵. <https://www.jsonrpc.org/specification>: JSON-RPC Specification